```r
library(ggplot2)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(tidyr)
library(lubridate)
```

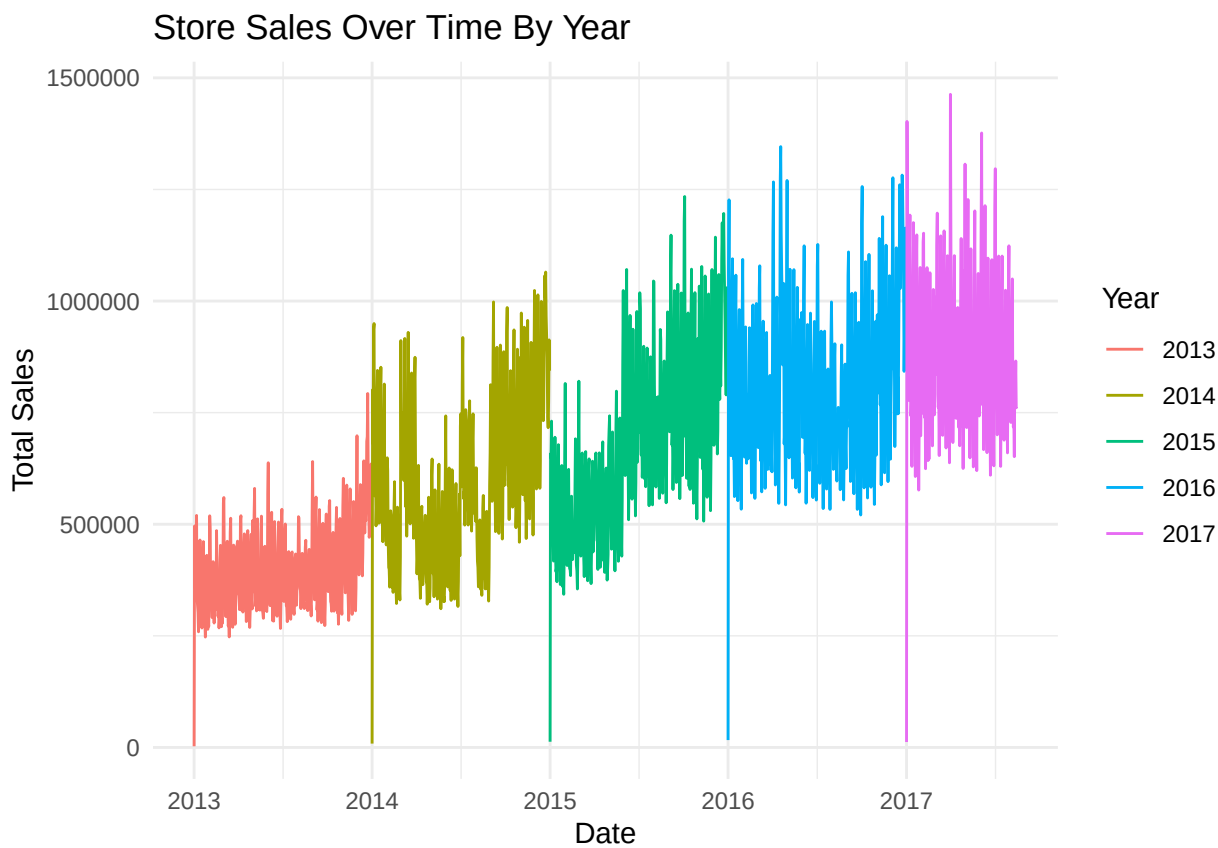
```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```r
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
train <- read.csv("../Data/train.csv")
oil <- read.csv("../Data/oil.csv")
test <- read.csv("../Data/test.csv")
```

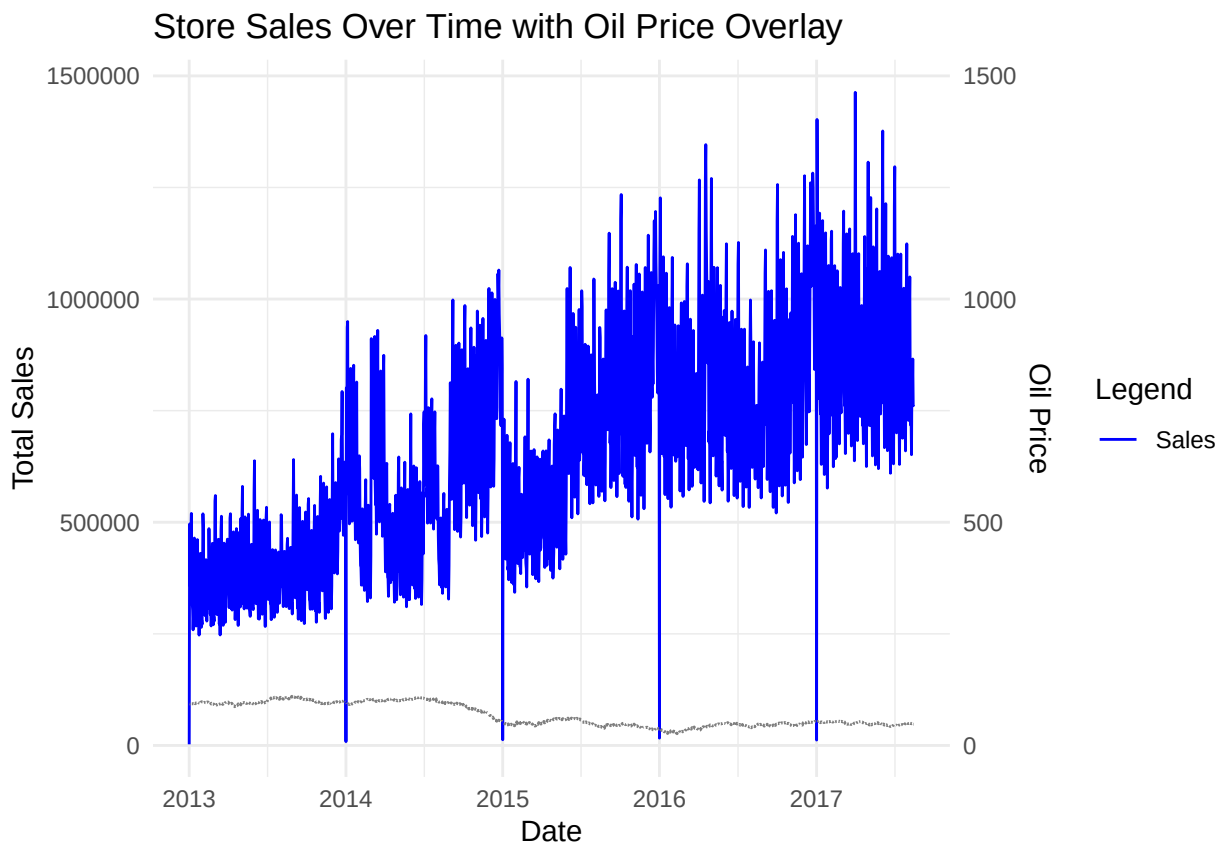
```r
sales_by_year <- train %>%
  mutate(date = as.Date(date)) %>%
  group_by(date) %>%
  summarise(total_sales = sum(sales)) %>%
  mutate(year = format(date, "%Y"))

ggplot(sales_by_year, aes(x = date, y = total_sales, color = year)) +
  geom_line() +
  labs(
    title = "Store Sales Over Time By Year",
    x = "Date",
    y = "Total Sales",
    color = "Year"
  ) +
  theme_minimal()
```



Can see a consistent upward trend in total sales by year.

```
#Garrett Sales Plot with Oil Price Overlay

train <- train %>%
  mutate(date = as.Date(date))

oil <- oil %>%
  mutate(date = as.Date(date))

# Joining Oil and Training Sets for Plotting Purposes

sales_oil_data <- train %>%
  group_by(date) %>%
  summarise(total_sales = sum(sales)) %>%
  left_join(oil, by = "date") %>%
  rename(oil_price = dcoilwtico)

# Scaling Oil Prices to Make Data Points Visible On Graph

ggplot(sales_oil_data, aes(x = date)) +
  geom_line(aes(y = total_sales, color = "Sales")) +
  geom_line(aes(y = oil_price * 1000, color = "Oil Price (Scaled")) +
  scale_y_continuous(
    name = "Total Sales",
    sec.axis = sec_axis(~ . / 1000, name = "Oil Price")
```

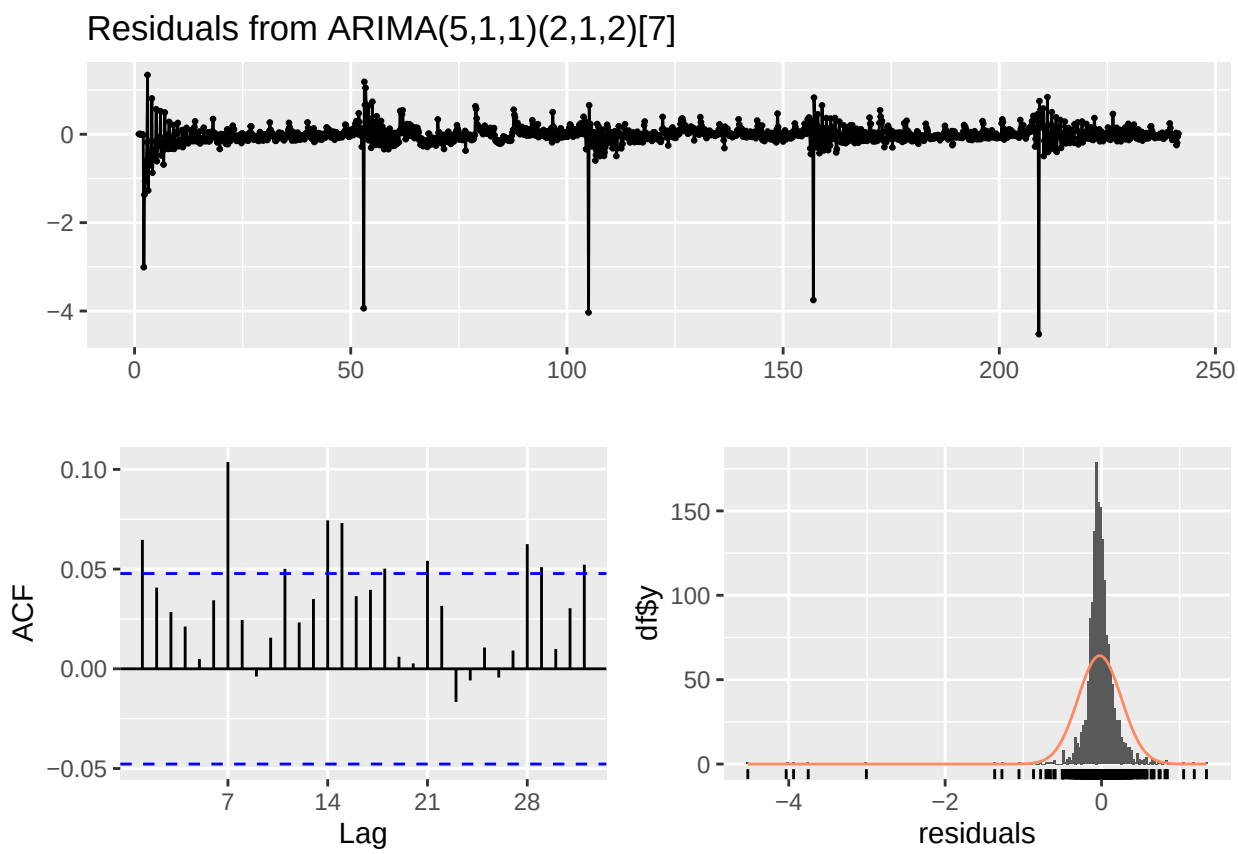
```
  ) +
  scale_color_manual(
    name = "Legend",
    values = c("Sales" = "blue", "Oil Price (Scaled)" = "red")
  ) +
  labs(
    title = "Store Sales Over Time with Oil Price Overlay",
    x = "Date"
  ) +
  theme_minimal()
```



Can see that higher oil prices in earlier years could have resulted in lower total sales because of factors such as transportation costs reducing overhead, and increased consumer spending because of higher disposable income values in the country.

```
train$date <- as.Date(train$date)
test$date <- as.Date(test$date)

train_agg <- train %>%
  group_by(date) %>%
  summarise(sales = sum(sales))

train_agg$sales_log <- log1p(train_agg$sales)

train_ts <- ts(train_agg$sales_log, frequency = 7)
```

```r
model <- Arima(train_ts, order = c(5, 1, 1), seasonal = c(2, 1, 2))

forecast_horizon <- length(unique(test$date))
forecasted_values <- forecast(model, h = forecast_horizon)

forecasted_sales <- expm1(forecasted_values$mean)

checkresiduals(model)
```




```
##
##  Ljung-Box test
##
## data:  Residuals from ARIMA(5,1,1)(2,1,2)[7]
## Q* = 50.354, df = 4, p-value = 3.046e-10
##
## Model df: 10.   Total lags used: 14
```

```r
test_dates <- unique(test$date)

proportions <- train %>%
  group_by(store_nbr, family) %>%
  summarise(avg_sales = mean(sales, na.rm = TRUE)) %>%
  mutate(proportion = avg_sales / sum(avg_sales)) %>%
  select(store_nbr, family, proportion)
```

```
## `summarise()` has grouped output by 'store_nbr'. You can override using the
## `.groups` argument.
```

```r
test_with_proportions <- test %>%
  left_join(proportions, by = c("store_nbr", "family"))

forecasted_sales_expanded <- test_with_proportions %>%
  group_by(date) %>%
  mutate(sales = forecasted_sales[which(test_dates == unique(date))] * proportion) %>%
  ungroup()

submission <- forecasted_sales_expanded %>%
  select(id, sales)

write.csv(submission, "submission4.csv", row.names = FALSE)
```

The code above was needed because ARIMA model was trained on aggregated data for the store_nbr and family. For the final submission file I needed to put each of these back into each individual row so that the submission file could be in the correct format. To do this I found that using past values of the proportion of family and store_nbr sales was the best way to ensure that the total sales value for each day would be evenly distributed throughout each day.

Submission 4 garnered an RMSLE of 4.01083, the best so far. Will be attempting to build on this model from this point forward.